

Tennis Match Simulation

Mia Emanuele

Table of contents

Design, build, test, explain, adapt, critique.....	2
Implementing Slam as two Python functions.....	2
Explanation of code design, incremental development, assumptions and testing.....	8
Designing my code	8
Assumptions	9
Final testing	10
Modifying the Python code to estimate two performance measures to a tennis coach that effectively summarise different aspects of a game of Slam	11
Table 1	21

Design, Build, Test, Explain, Adapt and Critique

Implementing Slam as two Python functions

The code I wrote to implement Slam, a dice-based tennis simulation, as two Python functions:

```
# =====  
# Simulation  
# =====  
  
import random  
  
def slam_dice_game():  
    """  
    Simulates a complete game of Slam using the official  
    Slam dice rules and returns the winning player.  
    """  
  
    print('\n-- New Point --')  
  
    keep_going = True  
  
    yellow_dice = random.randint(1,6)  
    print('Server rolls a', yellow_dice, 'with yellow dice')  
  
    if (yellow_dice == 1 or yellow_dice == 2):  
        keep_going = False  
        victor = 'server'  
        print('Ace winner, server wins point!')  
  
    elif (yellow_dice == 3):  
        keep_going = False  
        victor = 'receiver'  
        print('Double fault, receiver wins point!')  
  
    else:  
        print('Rally begins')
```

```

while (keep_going):

    blue_dice = random.randint(1,6)
    print('Reciever rolls a', blue_dice, 'with blue dice')

    if (blue_dice == 1):
        keep_going = False
        victor = 'receiver'
        print('In! Receiver wins point!')

    elif (blue_dice == 2):
        keep_going = False
        victor = 'server'
        print('Out! Server wins point!')

    else:
        print("Receiver hits in, rally continues")

if (keep_going):

    red_dice = random.randint(1,6)
    print('Server rolls a', red_dice, 'with red dice')

    if (red_dice == 1):
        keep_going = False
        victor = 'server'
        print('In! Server wins point!')

    elif (red_dice == 2):
        keep_going = False
        victor = 'receiver'
        print('Out! Receiver wins point!')

    else:
        print("Server hits in, rally continues")

return victor

```

```

# =====

# Tennis Scoring

# =====

def slam_dice_game_points():
    """
    Simulates a single point in a game of Slam and
    returns the winning player for that point.
    """

    print("\n-----")
    print('New game begins\n')

    points = {0: "0", 1: "15", 2: "30", 3: "40"}
    server_points = 0
    receiver_points = 0
    keep_going = True

    while (keep_going):

        victor = slam_dice_game()
        if (victor == 'server'):
            server_points += 1

        else:
            victor == 'receiver'
            receiver_points += 1

    # -----

    # Scoring if game reaches a deuce

```

```

# -----
if (server_points >= 3 and receiver_points >= 3):
    if (server_points == receiver_points):
        print('Score: Deuce (40-40)')

    elif (server_points == receiver_points + 1):
        print('Score: Server has advantage')

    elif (receiver_points == server_points + 1):
        print('Score: Receiver has advantage')

    elif (server_points >= receiver_points + 2):
        print('Server wins the game!')
        print('\n-- Game over --')
        keep_going = False

    elif (receiver_points >= server_points + 2):
        print('Receiver wins the game!')
        print('\n-- Game over --')
        keep_going = False

# -----

# Scoring before/if no deuce

# -----
else:
    if (server_points == 4):
        print('Server wins the game!')
        print('\n-- Game over --')
        keep_going = False

    elif (receiver_points == 4):
        print('Receiver wins the game!')
        print('\n-- Game over --')
        keep_going = False

    else:
        print('Score:',
              points[server_points], "-",
              points[receiver_points])
return

```

slam_dice_game_points()

Output including a deuce:

New game begins

-- New Point --

Server rolls a **6 with** yellow dice

Rally begins

Reciever rolls a **3 with** blue dice

Receiver hits **in**, rally continues

Server rolls a **1 with** red dice

In! Server wins point!

Score: **15 - 0**

-- New Point --

Server rolls a **2 with** yellow dice

Ace winner, server wins point!

Score: **30 - 0**

-- New Point --

Server rolls a **1 with** yellow dice

Ace winner, server wins point!

Score: **40 - 0**

-- New Point --

Server rolls a **3 with** yellow dice

Double fault, receiver wins point!

Score: **40 - 15**

-- New Point --

Server rolls a **3 with** yellow dice

Double fault, receiver wins point!

Score: **40 - 30**

-- New Point --

Server rolls a **3 with** yellow dice
Double fault, receiver wins point!
Score: Deuce (**40-40**)

-- New Point --

Server rolls a **3 with** yellow dice
Double fault, receiver wins point!
Score: Receiver has advantage

-- New Point --

Server rolls a **6 with** yellow dice
Rally begins
Receiver rolls a **1 with** blue dice
In! Receiver wins point!
Receiver wins the game!

-- Game over --

Output not including a deuce:

New game begins

-- New Point --

Server rolls a **5 with** yellow dice
Rally begins
Receiver rolls a **4 with** blue dice
Receiver hits **in**, rally continues
Server rolls a **3 with** red dice
Server hits **in**, rally continues
Receiver rolls a **2 with** blue dice
Out! Server wins point!
Score: **15 - 0**

-- New Point --

Server rolls a **2 with** yellow dice
Ace winner, server wins point!
Score: **30 - 0**

-- New Point --
Server rolls a 2 with yellow dice
Ace winner, server wins point!
Score: 40 - 0

-- New Point --
Server rolls a 5 with yellow dice
Rally begins
Receiver rolls a 4 with blue dice
Receiver hits in, rally continues
Server rolls a 2 with red dice
Out! Receiver wins point!
Score: 40 - 15

-- New Point --
Server rolls a 2 with yellow dice
Ace winner, server wins point!
Server wins the game!

-- Game over --

Explanation of code design, incremental development, assumptions and testing

Designing my code

Firstly, I ensured I fully understood the rules and mechanics of the Slam dice game before beginning the implementation. Based on this understanding, I selected an appropriate function name and added a descriptive docstring to explain its purpose. I then introduced the Boolean variable `keep_going` to control the flow of the game and determine when a point should end. Initially, I placed the `while (keep_going)` loop before implementing the yellow dice logic and then defined the conditions associated with each possible outcome of the server's serve. During testing, however, I realised this introduced a logical error because the yellow dice was being rolled repeatedly during a rally, whereas it should only be rolled once

at the start of each point. This highlighted the importance of validating the program logic throughout development.

For each possible outcome of the yellow dice, I updated the value of `keep_going` accordingly. If the server hit an ace or committed a double fault, the point ended immediately by setting `keep_going` to `False`; otherwise, play continued into a rally. I also incorporated descriptive print statements, such as *"Ace winner, server wins point!"* and *"Double fault, receiver wins point!"*, to provide clear storytelling throughout the simulation. At this stage, I tested the implementation by checking that `random.randint(1, 6)` generated valid dice rolls and by running the function to verify that each serve outcome behaved as expected.

Once the serve logic was functioning correctly, I implemented the rally by introducing the blue and red dice. The blue dice represented the receiver's shot, while the red dice represented the server's response. For each dice roll, I defined the corresponding outcomes and again used `keep_going` to determine whether the rally should continue or terminate. Additional print statements, including *"Receiver hits in, rally continues"* and *"Out! Server wins point!"*, were added to improve the readability of the simulation and allow the progress of each rally to be followed easily.

Following further testing, I identified two important logical errors. The first was that the yellow dice continued to be rolled during the rally because it had been incorrectly placed inside the loop. Relocating the `while (keep_going)` loop beneath the yellow dice conditions ensured that the serve occurred only once per point, accurately reflecting the rules of Slam. The second issue occurred when the receiver won the point using the blue dice; despite the point being over, the program still proceeded to roll the red dice. To resolve this, I introduced an additional `if (keep_going):` condition before the server's turn. This ensured that the rally terminated immediately once a winner had been determined, producing a more accurate simulation of the game.

After completing the point simulation, I developed a second function to implement the tennis scoring system. I began by creating variables to track the server's and receiver's points before adding conditions to update the score following each point. Initially, I implemented the standard scoring sequence before extending the logic to handle deuce and advantage scenarios. A dictionary was then introduced to convert numerical scores into the traditional tennis scoring format (0, 15, 30 and 40), making the output more representative of a real tennis game. Finally, I incorporated additional print statements to display each dice roll, point outcome, score update, and game status. These outputs not only satisfied the storytelling requirements of the assignment but also made it significantly easier to test and verify the correctness of the implementation.

Assumptions

There are several assumptions underlying the implementation of this simulation. Firstly, the model simulates a single game of Slam rather than a complete tennis set or match, allowing the analysis to focus solely on the mechanics of an individual game. Secondly, both the server and receiver are assumed to have equal ability, as the outcome of each point is determined entirely by rolling unbiased six-sided dice. In reality, differences in player skill, consistency and tactical ability would influence the probability of winning points.

Furthermore, the simulation does not account for factors that commonly influence real tennis performance, such as player fatigue, psychological pressure, momentum or crowd influence. Instead, each point is treated as an independent event with outcomes determined only by the predefined probabilities associated with the dice. Finally, I assumed that the scoring system followed the standard rules of tennis, with points progressing through 0, 15, 30 and 40, followed by deuce and advantage where necessary until one player wins by a two-point margin.

Final testing

Throughout the development process, I tested the code incrementally after implementing each major section to ensure the logic behaved as expected. Once the implementation was complete, I carried out additional testing to verify the overall correctness and reliability of the simulation.

Initially, I repeatedly executed the function using the following code:

```
# =====  
  
# Testing function  
  
# =====  
  
for i in range(20):  
    slam_dice_game_points()
```

Running multiple simulated games enabled me to verify that each game always terminated correctly with a single winner. I also checked that the yellow dice was rolled only once at the start of each point, that the blue and red dice alternated correctly during rallies, and that the score updated accurately after every point. Particular attention was given to ensuring that deuce and advantage situations occurred only when appropriate and that the game could only be won by a two-point margin following deuce.

I then investigated whether the probability of winning appeared reasonable by comparing the number of server and receiver victories across multiple simulations. Since the server always initiates each point, I expected the server to win slightly more frequently than the receiver, but not by an unrealistic margin. The following code was used to perform this check:

```
# =====  
  
# Probability of server winning  
  
# =====  
  
server_wins = 0  
receiver_wins = 0
```

```
for i in range(500):
    winner = slam_dice_game()

    if (winner == 'server'):
        server_wins += 1
    else:
        receiver_wins += 1

print("Server:", server_wins)
print("Receiver:", receiver_wins)
```

Output:

Server: 270

Receiver: 230

The results showed 270 server wins and 230 receiver wins across 500 simulated points. This distribution was consistent with my expectations, indicating that the simulation behaved realistically while preserving the slight advantage associated with serving first. Had the results shown an extreme imbalance, such as 400 server wins and only 100 receiver wins, this would have suggested that an error remained within the implementation or probability logic.

Modifying the Python code to estimate two performance measures to a tennis coach that effectively summarise different aspects of a game of Slam

I extended the Python code developed in Part (a) to estimate two performance measures that could provide meaningful insights to a tennis coach: serving dominance and average rally length. These measures summarise different aspects of player performance and demonstrate how simulation can be used to investigate characteristics of a tennis game beyond simply identifying the winner.

The first performance measure, serving dominance, was included to investigate how frequently the server wins the game. This provides an indication of the advantage associated with serving first and reflects the effectiveness of the serve throughout the simulation. A higher serving dominance may indicate stronger serving performance and a reduced likelihood of losing points through errors such as double faults. From a coaching perspective,

this measure helps assess whether serving provides a significant competitive advantage and may highlight areas where serving consistency could be improved.

The second performance measure was the average rally length, calculated across all rallies that occurred during the simulation. This measure provides an indication of whether points typically end quickly or develop into longer exchanges. Shorter average rallies suggest more aggressive play, with points being won or lost rapidly, whereas longer rallies indicate a greater emphasis on consistency and defensive play. Such information could help a coach tailor training sessions to reflect typical match characteristics, for example by focusing on attacking play to shorten rallies or endurance and consistency to sustain longer exchanges.

To further investigate the behaviour of the simulation, I modified the model to examine how these performance measures and the probability of the server winning changed when the game began from different starting scores. Specifically, I compared games starting at 15–0, where the server begins with a one-point advantage, and 0–15, where the receiver begins with the advantage. These scenarios enabled the simulation to explore how an early lead influences the overall outcome of the game and whether the measured performance indicators change under different match conditions.

The modifications required several additions to the original implementation. New variables were introduced to record the number of aces, double faults, rally lengths and rally counts, while dictionaries were used to store these statistics for each simulated game. The simulation was then executed repeatedly across a large number of games to calculate average performance measures and estimate the server's probability of winning under each starting condition. By extending the original simulation in this way, the model provides a more comprehensive analysis of tennis performance while demonstrating how probabilistic simulation can be used to investigate practical coaching questions.

```
import random
```

```
def slam_dice_game_mod():
```

```
    """
```

```
    Simulates a complete game of Slam while recording  
    performance measures including aces, double faults  
    and average rally length.
```

```
    """
```

```
    yellow_dice = random.randint(1, 6)
```

```
    victor = None
```

```
result_type = None # added variable
rally_length = 0 # added variable
keep_going = True

if yellow_dice == 1 or yellow_dice == 2:
    victor = 'server'
    result_type = 'ace'
    keep_going = False

elif yellow_dice == 3:
    victor = 'receiver'
    result_type = 'double_fault'
    keep_going = False

else:
    result_type = 'rally'

while keep_going:

    blue_dice = random.randint(1, 6)
    rally_length += 1

    if blue_dice == 1:
        victor = 'receiver'
        keep_going = False

    elif blue_dice == 2:
        victor = 'server'
        keep_going = False

    else:
        red_dice = random.randint(1, 6)
        rally_length += 1

        if red_dice == 1:
            victor = 'server'
            keep_going = False

        elif red_dice == 2:
            victor = 'receiver'
```

```

        keep_going = False

    return victor, result_type, rally_length # return different values


def slam_mod_points_and_stats():
    """
    Scoring for and tallying the outcomes of the
    modified game of Slam.
    """

    server_points = 0
    receiver_points = 0

    ace_count = 0 # new variable
    double_fault_count = 0 # new variable
    rally_total = 0 # new variable
    rally_points = 0 # new variable

    first_point_winner = None # new variable
    winner = None # new variable
    keep_going = True

    while keep_going:

        victor, result_type, rally_length = slam_dice_game_mod()

        if first_point_winner is None:
            first_point_winner = victor

        if result_type == 'ace':
            ace_count += 1 # counting aces

        if result_type == 'double_fault':
            double_fault_count += 1 # counting double faults

        if rally_length > 0:
            rally_total += rally_length

```

```

    rally_points += 1 # counting rallies

    if victor == 'server':
        server_points += 1
    else:
        receiver_points += 1

    if server_points >= 4 and server_points >= receiver_points + 2:
        winner = 'server'
        keep_going = False

    elif receiver_points >= 4 and receiver_points >= server_points + 2:
        winner = 'receiver'
        keep_going = False

game_stats = { # new dictionary to show stats
    'aces': ace_count,
    'double_faults': double_fault_count,
    'rally_total': rally_total,
    'rally_points': rally_points
}

return winner, first_point_winner, game_stats

#####

def simulate_games(total_games):
    """Running the game and calculating the averages
    of the two performance measures outcomes."""

    games_15_0 = 0 # 15-0 stats
    server_wins_15_0 = 0
    aces_15_0 = 0
    df_15_0 = 0
    rally_len_15_0 = 0
    rally_pts_15_0 = 0

```

```

games_0_15 = 0 # 0-15 stats
server_wins_0_15 = 0
aces_0_15 = 0
df_0_15 = 0
rally_len_0_15 = 0
rally_pts_0_15 = 0

for i in range(total_games):

    winner, first_point, stats = slam_mod_points_and_stats()

    # -----
    # If game starts at 15-0
    # -----
    if first_point == 'server':

        games_15_0 += 1

        if winner == 'server':
            server_wins_15_0 += 1

        aces_15_0 += stats['aces']
        df_15_0 += stats['double_faults']
        rally_len_15_0 += stats['rally_total']
        rally_pts_15_0 += stats['rally_points']

    # -----
    # If game starts at 0-15
    # -----
    else:

        games_0_15 += 1

        if winner == 'server':
            server_wins_0_15 += 1

        aces_0_15 += stats['aces']
        df_0_15 += stats['double_faults']

```



```

    rally_len_0_15 += stats['rally_total']
    rally_pts_0_15 += stats['rally_points']

# -----

# Calculating averages for performance measures

# -----
if rally_pts_15_0 > 0:
    avg_rally_15_0 = rally_len_15_0 / rally_pts_15_0
else:
    avg_rally_15_0 = 0

if rally_pts_0_15 > 0:
    avg_rally_0_15 = rally_len_0_15 / rally_pts_0_15
else:
    avg_rally_0_15 = 0

if games_15_0 > 0:
    server_dom_15_0 = server_wins_15_0 / games_15_0
else:
    server_dom_15_0 = 0

if games_0_15 > 0:
    server_dom_0_15 = server_wins_0_15 / games_0_15
else:
    server_dom_0_15 = 0

print('Total games simulated:', total_games)
print('\n-- Games Starting 15-0 (Server has 1-point advantage) --')
print('Games:', games_15_0)
print('Server win rate:', server_dom_15_0)
print('Total aces:', aces_15_0)
print('Total double faults:', df_15_0)
print('Average rally length:', avg_rally_15_0)

print('\n-- Games Starting 0-15 (Receiver has 1-point advantage) --')
print('Games:', games_0_15)
print('Server win rate:', server_dom_0_15)
print('Total aces:', aces_0_15)

```

```
print('Total double faults:', df_0_15)
print('Average rally length:', avg_rally_0_15)

return
```

```
simulate_games(10000)
```

Output:

Total games simulated: 10000

-- Games Starting 15-0 (Server has 1-point advantage) --

Games: 5873

Server win rate: 0.8225778988591861

Total aces: 13690

Total double faults: 4994

Average rally length: 2.9427080410751363

-- Games Starting 0-15 (Receiver has 1-point advantage) --

Games: 4127

Server win rate: 0.5340440998303853

Total aces: 8231

Total double faults: 5891

Average rally length: 3.04425618434429

The code for simulating games starting at 0-0:

```
def simulate_games(total_games):  
    """  
    Scoring for and tallying the outcomes of the  
    modified game of Slam.  
    """  
  
    server_wins_0_0 = 0  
  
    aces_0_0 = 0  
    df_0_0 = 0  
    rally_len_0_0 = 0  
    rally_pts_0_0 = 0  
  
    for i in range(total_games):  
  
        winner, first_point, stats = slam_mod_points_and_stats()  
  
        if winner == 'server':  
            server_wins_0_0 += 1  
  
        aces_0_0 += stats['aces']  
        df_0_0 += stats['double_faults']  
        rally_len_0_0 += stats['rally_total']  
        rally_pts_0_0 += stats['rally_points']  
  
        # -----  
  
        # Compute averages  
  
        # -----  
        if rally_pts_0_0 > 0:  
            avg_rally_length = rally_len_0_0 / rally_pts_0_0  
        else:  
            avg_rally_length = 0  
  
    server_win_rate = server_wins_0_0 / total_games  
  
    print('Total games simulated:', total_games)  
    print('\n-- Games Starting from 0-0 --')
```

```
print('Server win rate:', server_win_rate)
print('Total aces:', aces_0_0)
print('Total double faults:', df_0_0)
print('Average rally length:', avg_rally_length)

return
```

```
simulate_games(10000)
```

Output:

Total games simulated: 10000

-- Games Starting from 0-0 --

Server win rate: 0.7041

Total aces: 21844

Total double faults: 10838

Average rally length: 3.005463813680901

Calculating the probability of a server winning a game, aces per game, double faults per game and length of rally per game:

```
print('Aces per game (0-0):',
      21844/10000)
print('Double faults per game (15-0):',
      10838/10000)

print('\nAces per game (15-0):',
      13690/5873)
print('Double faults per game (15-0):',
      4994/5873)

print('\nAces per game (0-15):',
      8231/4127)
print('Double faults per game (0-15):',
      5891/4127)
```

Output:

Aces per game (0-0): **2.1844**

Double faults per game (15-0): **1.0838**

Aces per game (15-0): **2.331006300017027**

Double faults per game (15-0): **0.8503320279243998**

Aces per game (0-15): **1.994426944511752**

Double faults per game (0-15): **1.427429125272595**

Table 1

Summary of the server's win probability and performance measures under different starting score conditions.

Starting Score	Server Win Probability	Aces per Game	Double Faults per Game	Average Rally Length
0-0	~70%	~2.18	~1.08	~3.01
15-0	~82%	~2.33	~0.85	~2.94
0-15	~53%	~1.99	~1.43	~3.04

The summary table above highlights several key findings relating to serving dominance and the selected performance measures within the Slam simulation. As shown by both the simulation outputs and the summary table, the probability of the server winning the game is highest when the server begins with a one-point advantage (15-0). This outcome is expected, as the server requires fewer additional points to secure victory.

The most significant observation, however, is the substantial reduction in the server's probability of winning when the game begins at 0-15. Although this may appear intuitive, it demonstrates how an early point can considerably influence the overall probability of winning the game. Within the context of the simulation, the initial score has a noticeable effect on the likelihood of success, illustrating how even a small advantage can influence the final outcome.

The performance measures also reveal several interesting trends. The average number of aces per game is slightly higher when the server starts with a one-point advantage, whereas the average number of double faults is greatest when the server begins one point behind. This suggests that starting from a disadvantaged position is associated with a greater number of serving errors within the simulation, while beginning with an advantage results in more successful serves.

In contrast, the average rally length remains relatively consistent across all three starting scenarios, varying only slightly between approximately three shots per rally. Games beginning at 15–0 produce marginally shorter rallies, whereas those beginning at 0–15 produce marginally longer rallies. Although these differences are small, they suggest that the starting score has a much greater influence on the probability of winning than on the duration of individual rallies.

Overall, the simulation demonstrates that an early point advantage substantially increases the server's likelihood of winning the game, while only having a limited effect on rally length. These findings illustrate how simulation can be used to investigate the relationship between initial conditions and performance measures, providing quantitative evidence that may support coaching analysis and strategic decision-making.